國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab 03

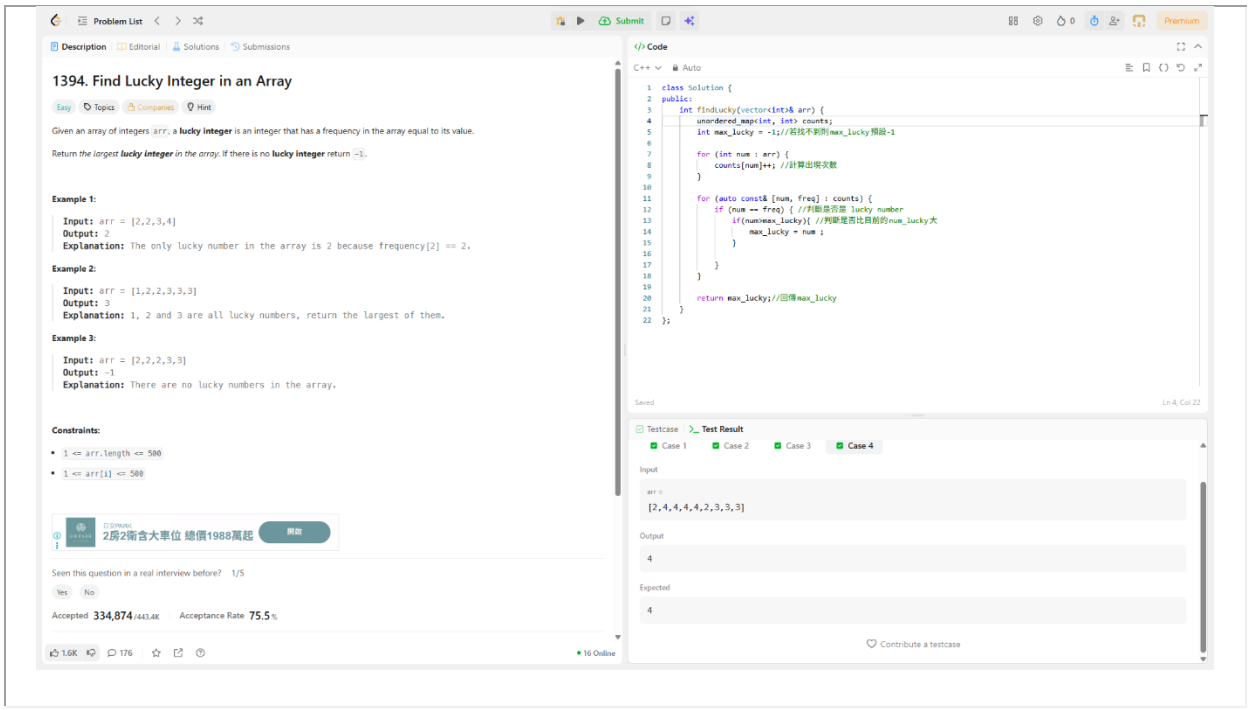
姓名：黃聖傑

學號：114310140

Lab01-Q1.

(1) Solve LeetCode 1394 and (2) discuss the time complexity.

Code



Problem List

Description | Editorial | Solutions | Submissions

1394. Find Lucky Integer in an Array

Easy | Topics | Companies | Hint

Given an array of integers `arr`, a **lucky integer** is an integer that has a frequency in the array equal to its value.

Return *the largest* ***lucky integer*** *in the array.* If there is no **lucky integer** return `-1`.

Example 1:

```
Input: arr = [2,2,3,4]
Output: 2
Explanation: The only lucky number in the array is 2 because frequency[2] == 2.
```

Example 2:

```
Input: arr = [1,2,2,3,3,3]
Output: 3
Explanation: 1, 2 and 3 are all lucky numbers, return the largest of them.
```

Example 3:

```
Input: arr = [2,2,2,3,3]
Output: -1
Explanation: There are no lucky numbers in the array.
```

Constraints:

- `1 <= arr.length <= 500`
- `1 <= arr[i] <= 500`

DEEPWIKI 2房2衛含大車位 總價1988萬起 開啟

Seen this question in a real interview before? 1/5

Yes | No

Accepted 334,874/443.4K | Acceptance Rate 75.5%

1.6K | 176 | 16 Online

Submit | Premium

Code

C++ | Auto

```cpp
class Solution {
public:
    int findLucky(vector<int>& arr) {
        unordered_map<int, int> counts;
        int max_lucky = -1;//若找不到則max_lucky預設-1

        for (int num : arr) {
            counts[num]++; //計算出現次數
        }

        for (auto const& [num, freq] : counts) {
            if (num == freq) { //判斷是否是 lucky number
                if(num>max_lucky){ //判斷是否比目前的num_lucky大
                    max_lucky = num ;
                }

            }
        }

        return max_lucky;//回傳max_lucky
    }
};
```

Saved | Ln 4, Col 22

Testcase | Test Result

Case 1 | Case 2 | Case 3 | Case 4

Input

arr =

[2,4,4,4,4,2,3,3,3]

Output

4

Expected

4

Contribute a testcase

Discussion Section

討論時間複雜度：

Total operations:

n(access array)+1(assignment max_lucky)+n(operation:+)+2n(Comparisons)+n(assignment max_lucky)+1(return)=5n+2

Therefore, O(n) complexity